Raspberry Pi Pico 2 User Guide

User guide · Support · Ask a Question

Raspberry Pi Pico 2 RP2350 Pico ...

Price: 39.0 RS
Brand: Raspberry Pi
Model: Raspberry Pi Pico 2

(Tutorial Navigation)
Overview
Introduction
Features
Pinout Definition
Dimensions
Software Environment Debugging
Examples
External LED Example
Traffic Light System Examples
Burglar Alarm LED Buzzer Examples
Potentiometer Example
MEMS Example
LCD1602 I2C Example
Resource
Firmware Library
Example Demos
Official Document
Pico2
User Manual
Schematic & Datasheet
Related Resources
Raspberry Pi Open Source Demos
Development Software
FAQ
Questions/Wish are the frequent questions for developing with the Raspberry Pi Pico 2!
Support

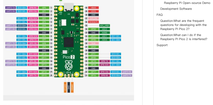
Overview

Introduction

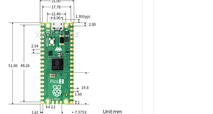
Raspberry Pi Pico 2 is a low-cost, high-performance microcontroller board with flexible digital interfaces. Key features include:

- RP2350 microcontroller chip designed by Raspberry Pi in the United Kingdom
- Dual Arm Cortex-M33 processor and dual-core Hazard3 RISC-V processor, flexible clock running up to 150 MHz
- 520kB of SRAM, and 4MB of on-board flash memory
- Castellated module allows soldering direct to carrier boards
- USB 1.1 host and device support
- Low power sleep and dormant modes
- Drag-and-drop programming using mass storage over USB
- 26 x multi-function GPIO pins
- 2 x SPI, 2 x I2C, 2 x UART, 3 x 12-bit ADC, 24 x controllable PWM channels
- Accurate clock and timer on-chip
- Temperature sensor
- 12 x Programmable I/O (PIO) state machines for custom peripheral support.

Pinout Definition



Dimensions

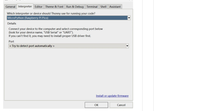


Software Environment Debugging

- To facilitate the development of Pico 2 boards using MicroPython on the computer, it is recommended to download Thonny IDE.
- Download Thonny IDE and install it to disks:
  - Thonny IDE download link (website)
  - Thonny IDE download
  - Thonny Python IDE Windows V3.3.6#
- After installing, please configure the language and the environment for the first time. Note that we should choose the Raspberry Pi option in the board environment.

- Click OK to return to the main interface of Thonny, download the Firmware Manual of Pico, and then click the drop button to display the currently used environment in the lower menu.

- Configure the MicroPython environment and select the Pico 2 port.



- First connect the Raspberry Pi Pico 2 to the computer, with click on the "Interpreter" option in the lower right corner of Thonny — select configure an interpreter.
- In the pop-up window, select MicroPython (Raspberry Pi Pico), and select the port corresponding port.

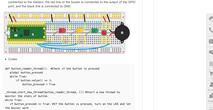
- Click OK to return to the main interface of Thonny, disconnect the Firmware Manual of Pico, and then click the drop button to display the currently used environment in the lower menu.
- Click to download firmware factory method in windows. Press and hold the BOOT button and connect to the computer, release the BOOT button, and a removable disk will appear on the computer and copy the firmware library UF2.
- Click on the Thonny IDE interface, the Pico 2 will be detected in windows. After connecting to the computer, press the BOOT key and RESET key at the same time, first release the RESET key and then release the BOOT key, the computer will appear in a removable disk, the firmware library can be copied into it. (Pico2 can now be used in the same way)

Examples

- Download Raspberry-Pi-Pico-Basic-V3-Demo# to your Raspberry-Pi and test.

External LED Example

- Connect the hardware as in the picture below. Connect the Pico 2 to the Micro USB Connector of the PC. Open the python file of Lesson 1 in the folder of example with Thonny. Run the example, and you will find that the red LED is flashing.
- Note: The length of the connection line in the picture is too long, the negative should be shortened to ensure the stability of the power supply. If the power supply is insufficient (connected to USB), the positive should be connected to the GPIO output port must be connected to the positive of the LED, and the negative must be connected to the ground of the GPIO port, and the black line is connected to GND.

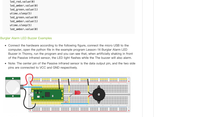


- Codes

```
led.value(1) # machine.Pin(15, machine.Pin.OUT) #Set GPIO to output Mode
while True:
    print("LED ON...")
    time.sleep(1) #Change the LED every 1 seconds.
    led.value(0)
```

Traffic Light System Examples

- Connect the hardware according to the following figure, connect the micro USB to the computer. Open the python file in the example program (Lesson 2) in Pico-Basic-V3-Demo, run the program you can see the normal operation of the traffic light area, when the button is pressed, the traffic light system is in the flashing state.
- Note: The length of the connection line in the picture is too long, the negative should be connected to GND, the positive should be connected to the GPIO output port must be connected to the positive of the LED, and the negative must be connected to the ground of the GPIO port, and the black line is connected to GND.

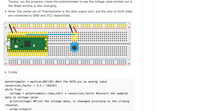


- Codes

```
led_pin1,led_pin2,led_pin3 #Interrupt handler function, buzzer sounds, led blinks regularly
button = machine.Pin(20, machine.Pin.IN,machine.Pin.PULL_UP)
for i in range(10):
    time.sleep(0.1)
    print("Waiting...")
    print(button.value())
button.irq(lambda_=button_irq,trigger=machine.Pin.IRQ_FALLING, handler=irq_handler)#Attach the interrupt, when the keys sensor detects an edge(falling edge) will be handled by the function irq_handler(button_irq)
while True: #Enterpen the status of the LED every 5 seconds when there are no interrupts
    time.sleep(5)
    print("Waiting...")
```

Burglar Alarm LED Buzzer Examples

- Connect the hardware according to the following figure, connect the micro USB to the computer. Open the python file in the example program (Lesson 3) in Pico-Basic-V3-Demo, run the program you can see the LED and buzzer flashing alternately, when the button is pressed, the LED and buzzer stop flashing and the LED is always on.
- Note: The center pin of the Passive infrared sensor is the data output pin, and the two side pins are connected to VCC and GND respectively.



- Codes

```
led_pin1,led_pin2,buzzer #Check if the button is pressed
button_pin=machine.Pin(20,
    machine.Pin.IN,
    machine.Pin.PULL_UP)
button_pin.irq(handler=irq_handler,trigger=1) #Enterpen a new thread to monitor the state of button
while True:
    if button_pin.value() == 0:
        print("button pressed !!!")
        print("button_pin.value()",button_pin.value())
        for buzzer in range(10):
            led_pin1.value(1)
            led_pin2.value(1)
            buzzer.value(1)
            time.sleep(0.1)
            led_pin1.value(0)
            led_pin2.value(0)
            buzzer.value(0)
            time.sleep(0.1)
        led_pin1.value(1)
        led_pin2.value(1)
        buzzer.value(0)
        time.sleep(0.1)
        led_pin1.value(0)
        led_pin2.value(0)
        buzzer.value(0)
        time.sleep(0.1)
    else:
        led_pin1.value(0)
        led_pin2.value(0)
        buzzer.value(0)
        time.sleep(0.1)
```

Potentiometer Example

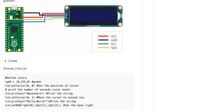
- Connect the hardware according to the following diagram, connect the micro USB to the computer. Open the python file in the example program (Lesson 4) Potentiometer in Pico-Basic-V3-Demo, run the program you can see the LED brightness changes from a dim to the Great brightness is also changing.
- Note: The center pin of Potentiometer is the data output port, and the pins on both sides are connected to VCC and GND respectively.

- Codes

```
potentiometer = machine.ADC(26) #Set the GPIO pin as analog input
while True:
    time.sleep(0.1)
    print("potentiometer.read_u16()",potentiometer.read_u16())#Read the analog output data by voltage value
    led.duty_u16(potentiometer.read_u16()) #Change according to the sliding resistor
    time.sleep(0.1)
```

MEMS Example

- Connect the hardware according to the following figure, connect the micro USB to the computer. Open the python file in the example program (Lesson 5) Potentiometer in Pico-Basic-V3-Demo, and run the program to see the RGB colors of blue, red, green, and white in one loop.



- Code

```
#This code uses the state machine. The following code is a decrease#
#Note:We use initialize the hardware, set the pin level, etc.
#Note:Because the state machine is used, the pin level is set at low level on time by default.
#
machine.Pin(27).off() #We set LED to be off.
#mqpin1.on() #If it is the code lower to here if a 1 s.
machine.Pin(22,machine.Pin.OUT).on(), machine.Pin(23,machine.Pin.OUT), machine.Pin(24,
machine.Pin.OUT)
#Define the variable
#
#Define the variable
#
#Define the variable
#
```